

MACHINE LEARNING · ENSEMBLE METHODS

Gradient Boosting Regression

A step-by-step walkthrough: predicting house prices from size, one small correction at a time.

MADE BY : Nancy Ahmed

STEP 1

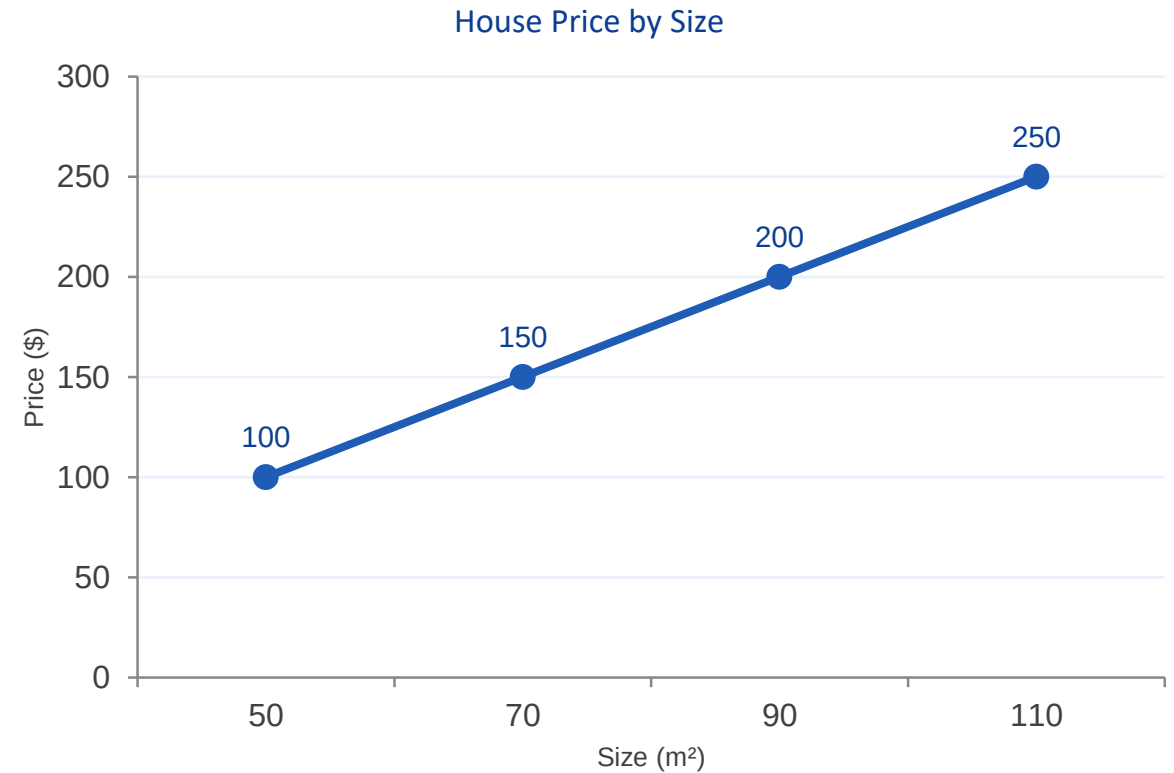
Our Dataset

We start with four houses. The goal is to predict price from size.

House Size (m ²)	Price (\$)
50	100
70	150
90	200
110	250

Notice: price rises by \$50 for every 20 m² — a clean linear pattern, useful for checking our model later.

Size vs. Price



STEP 2

Initial Prediction

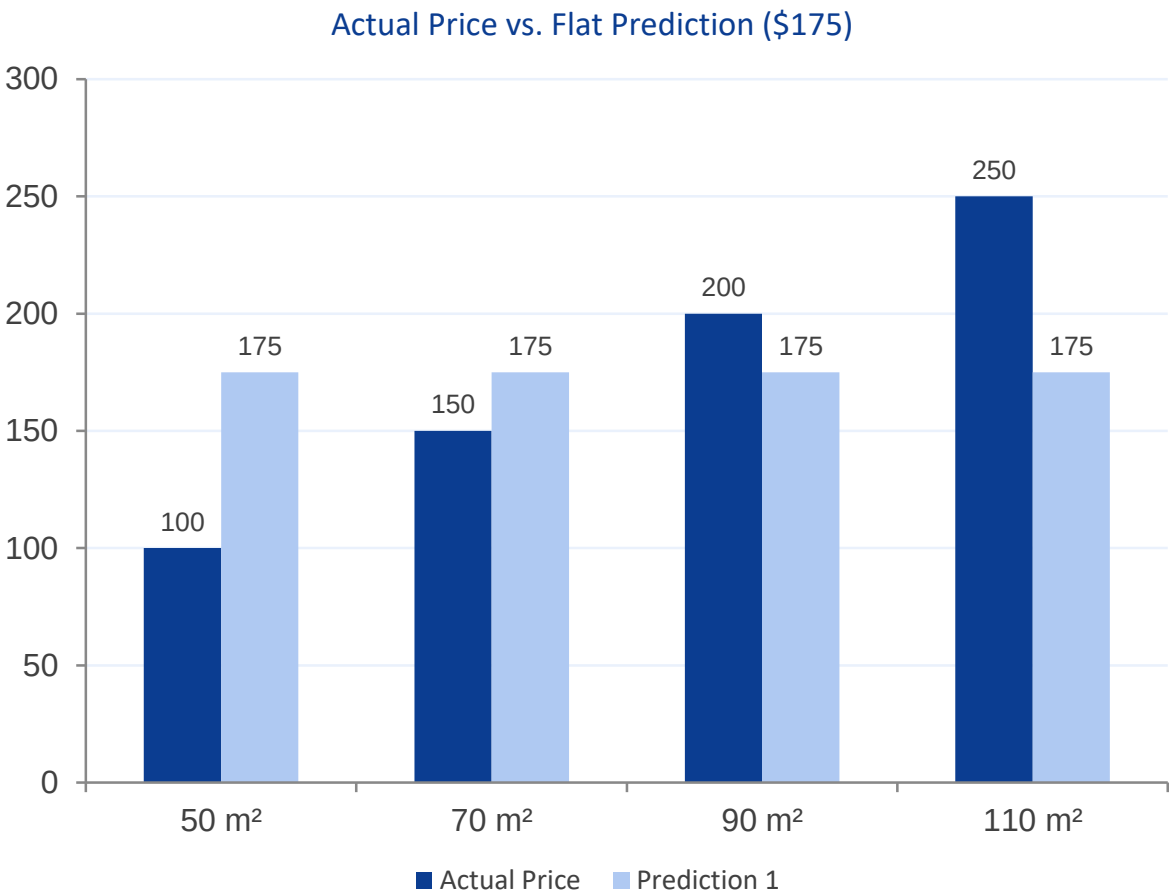
The first model doesn't build a tree — it simply predicts the average price for every house.

Average = $(100+150+200+250) / 4$

= 175

Every house is given the exact same starting prediction: \$175.

Actual	Prediction 1
100	175
150	175
200	175
250	175



STEP 3

Calculate Residuals

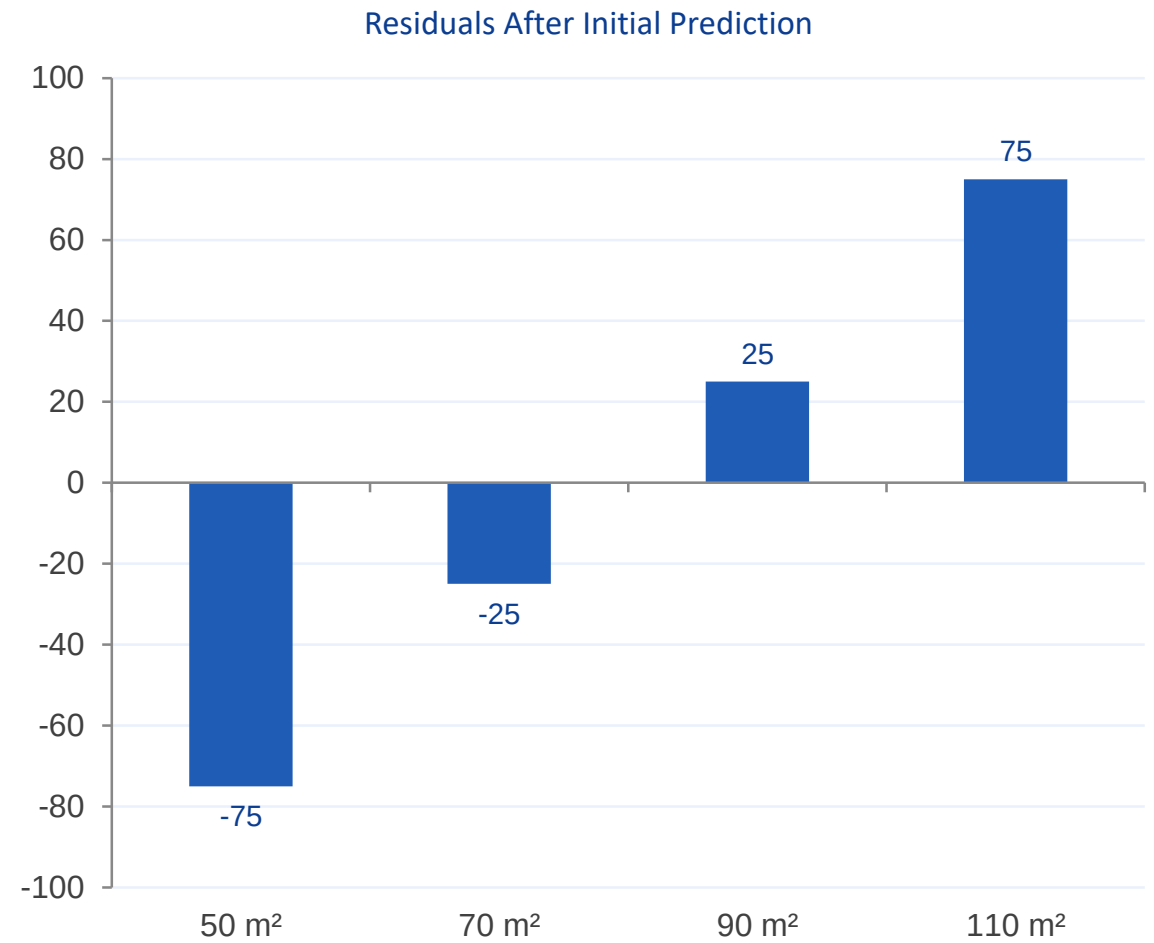
Residual = Actual – Prediction

The residual tells us how much the first prediction was wrong.

Actual	Prediction	Residual
100	175	-75
150	175	-25
200	175	25
250	175	75

Negative → predicted too high

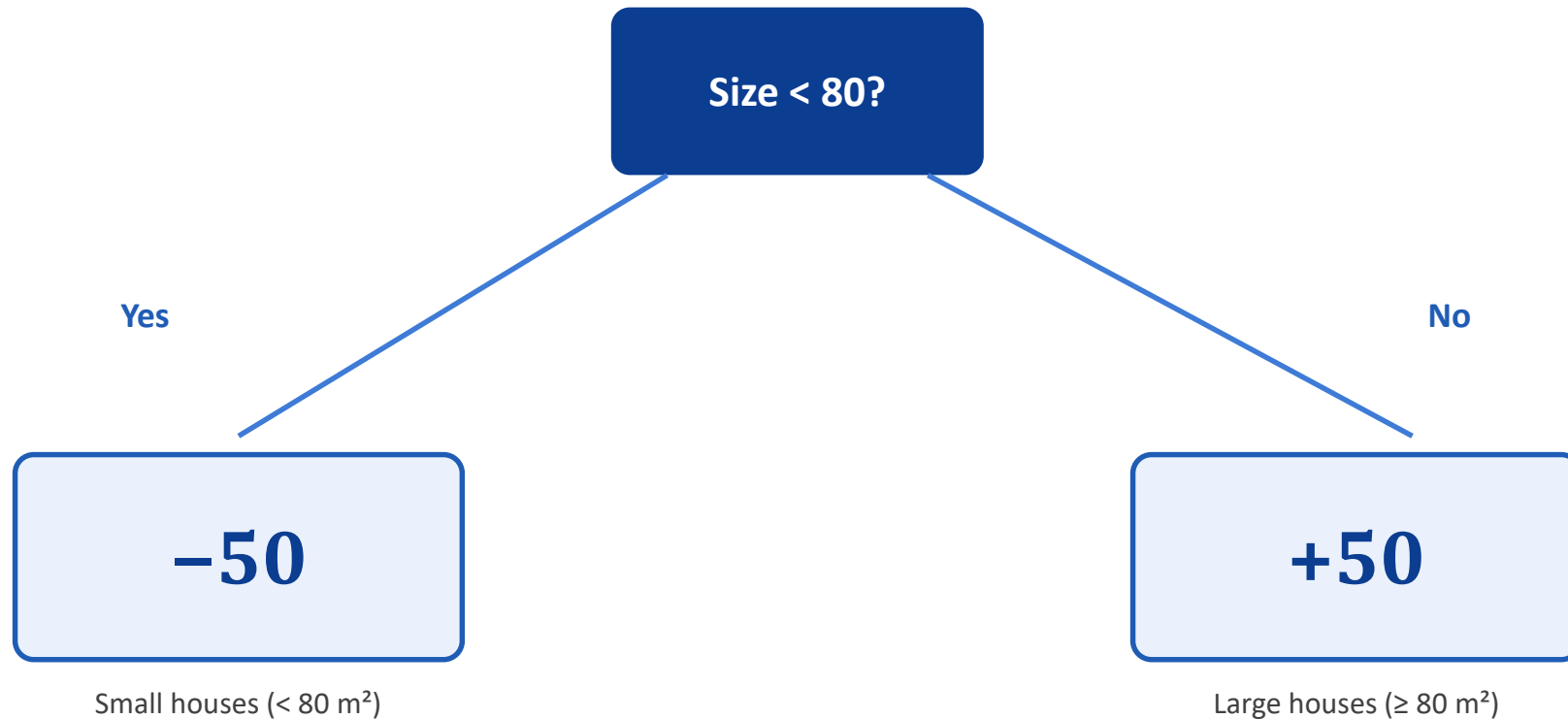
Positive → predicted too low



STEP 4

Train Tree 1

Tree 1 doesn't predict prices — it predicts the residuals from Step 3.



Meaning: small houses need a -50 correction; large houses need a +50 correction. *The tree predicts the residual, not the price.*

STEP 5

Update Predictions

New Prediction = Old Prediction + **0.1** × Tree Output

With a learning rate of 0.1, we add only 10% of each tree's correction — not the full amount.

House	Old Prediction	Tree Output	Correction (×0.1)	New Prediction
50 m ²	175	-50	-5	170
70 m ²	175	-50	-5	170
90 m ²	175	+50	+5	180
110 m ²	175	+50	+5	180

The predictions have improved — but they're still not perfect. As training continues, each new tree will chip away a little more.

Why so small? A learning rate of 0.1 means each tree only nudges predictions 10% of the way — this keeps the model from overreacting to any single tree's guess.

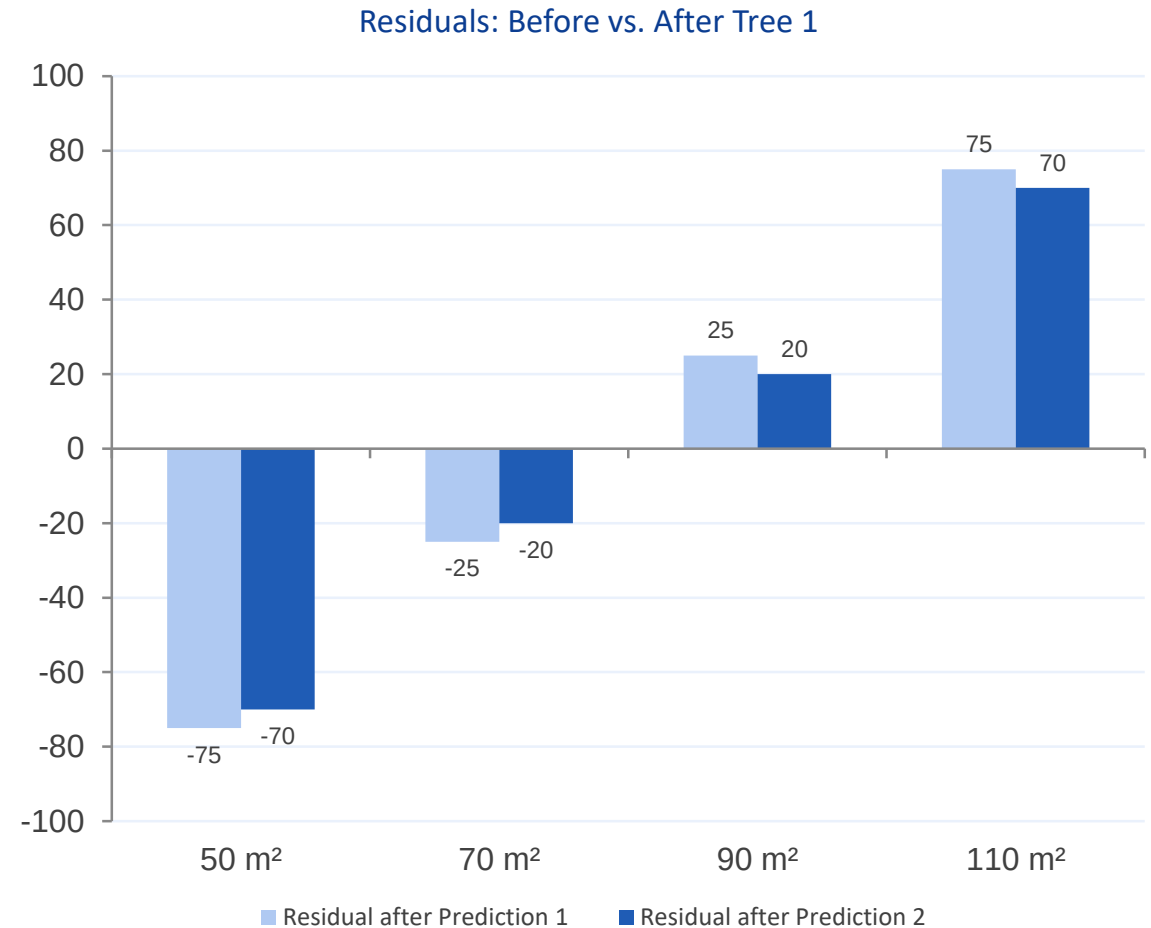
STEP 6

Calculate New Residuals

Residual = Actual – New Prediction

Actual	Prediction	Residual
100	170	-70
150	170	-20
200	180	20
250	180	70

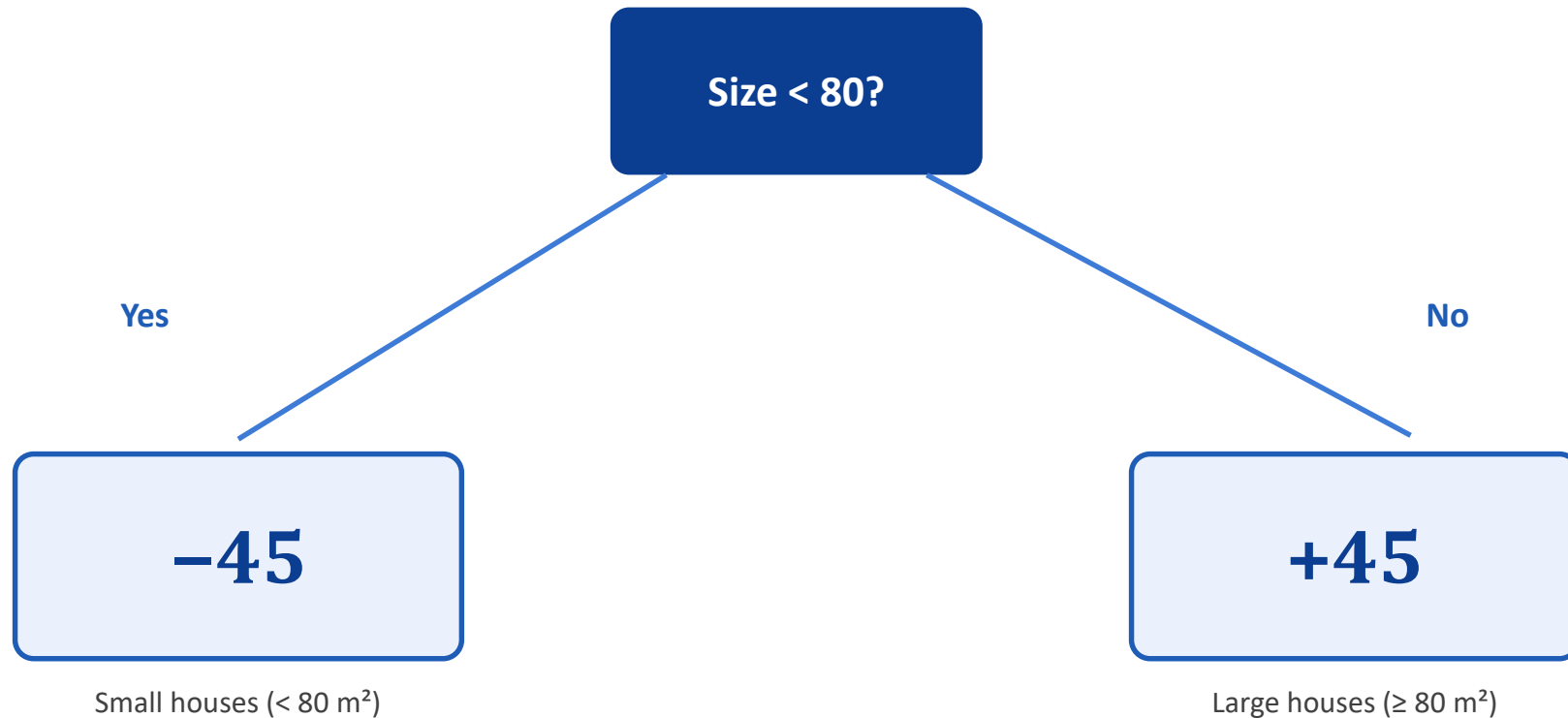
Progress check: residual magnitudes shrank from ± 75 / ± 25 down to ± 70 / ± 20 — smaller, but not gone yet.



STEP 7

Train Tree 2

Tree 2 is trained on the new residuals from Step 6. In this simplified example, it learns the same split.



Meaning: small houses need a -45 correction; large houses need a +45 correction. *The tree predicts the residual, not the price.*

STEP 8

Update Again

Again, we add only 10% of Tree 2's output to the previous prediction.

House	Previous Prediction	Tree 2 Output	Correction ($\times 0.1$)	New Prediction
50 m ²	170	-45	-4.5	165.5
70 m ²	170	-45	-4.5	165.5
90 m ²	180	+45	+4.5	184.5
110 m ²	180	+45	+4.5	184.5

Pattern so far: 175 $\rightarrow$ 170 $\rightarrow$ 165.5 $\rightarrow$... each new tree nudges predictions \$5 closer to the true price. With enough trees, predictions converge toward the actual values.

Why the Learning Rate Matters

The learning rate controls how much we trust each new tree's correction.

LARGE LEARNING RATE

e.g., 1.0

- ✓ Big corrections, faster learning
- ✗ Higher risk of overfitting
- ✗ Can overshoot the true values

SMALL LEARNING RATE

e.g., 0.1

- ✓ Small, careful corrections
- ✓ Usually better generalization
- ✗ Needs more trees to converge

How Gradient Boosting Works

01 **Start simple**

The first prediction is just the average of all target values.

02 **Predict the error**

Each new tree learns to predict the previous model's residuals — not the price itself.

03 **Add cautiously**

A learning rate (e.g., 0.1) scales down each tree's contribution to avoid overreacting.

04 **Repeat and converge**

Residuals shrink with every tree, and predictions gradually approach the true values.